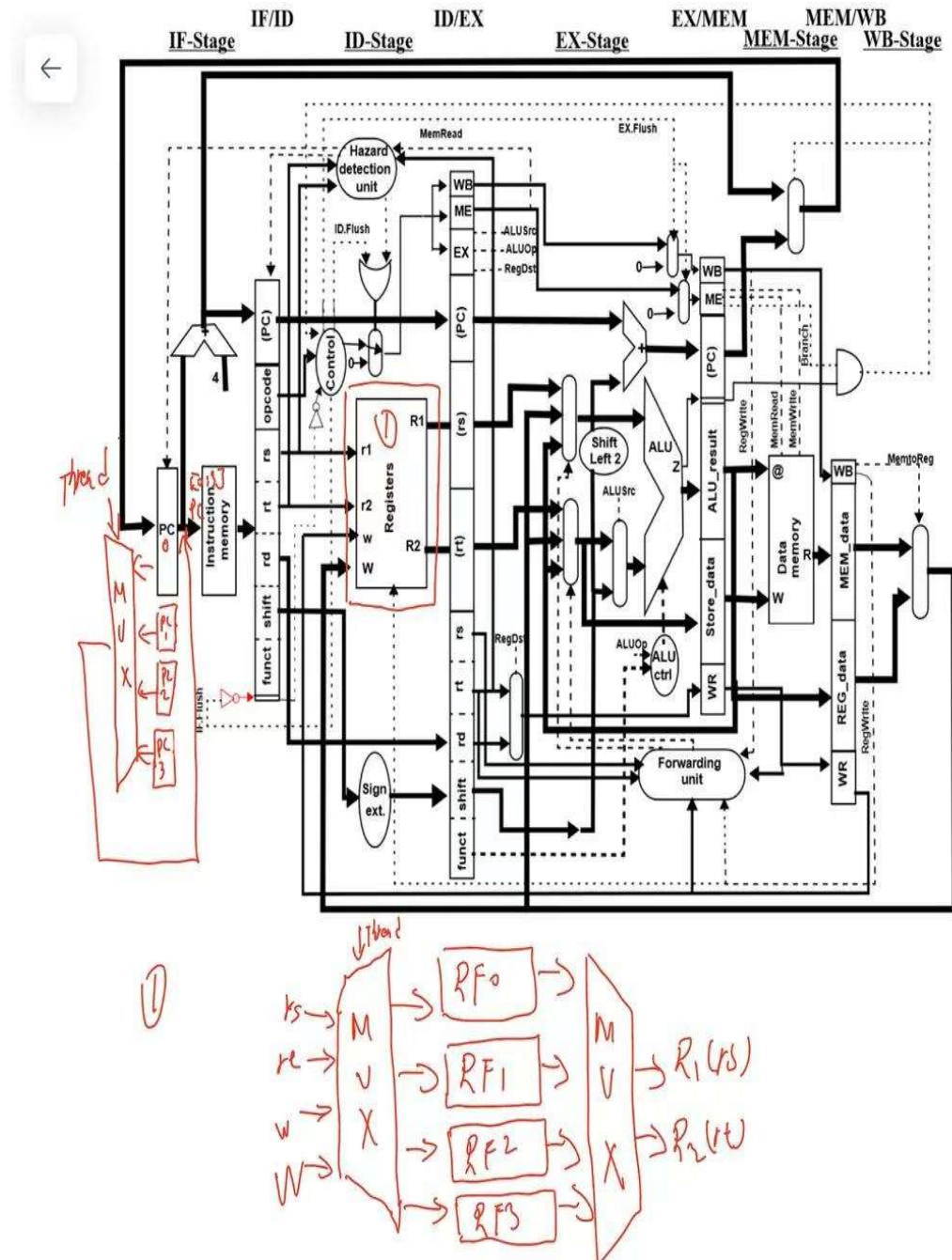


Lab9 Report

Remote repository address : <https://github.com/hzhang2422/EE-533-Team4>

Members: Yijie Zhou, Jiahe Wu, Haoyang Zhang, Bohan Fang

Datapath design:



Hardware Processor:

```
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
// Module: processor.v
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
`timescale 1ns/1ps

module processor
#(
    parameter DATA_WIDTH = 64,
    parameter CTRL_WIDTH = DATA_WIDTH/8,
    parameter UDP_REG_SRC_WIDTH = 2
)
(
    // --- Packet Data interface
    input  [DATA_WIDTH-1:0]      in_data,
    input  [CTRL_WIDTH-1:0]      in_ctrl,
    input                               in_wr,
    output                              in_rdy,

    output [DATA_WIDTH-1:0]      out_data,
    output [CTRL_WIDTH-1:0]      out_ctrl,
    output                              out_wr,
    input                               out_rdy,

    // --- Register interface
    input                               reg_req_in,
    input                               reg_ack_in,
    input                               reg_rd_wr_L_in,
    input  [`UDP_REG_ADDR_WIDTH-1:0]   reg_addr_in,
    input  [`CPCI_NF2_DATA_WIDTH-1:0]   reg_data_in,
    input  [UDP_REG_SRC_WIDTH-1:0]      reg_src_in,

    output                              reg_req_out,
    output                              reg_ack_out,
    output                              reg_rd_wr_L_out,
    output  [`UDP_REG_ADDR_WIDTH-1:0]   reg_addr_out,
    output  [`CPCI_NF2_DATA_WIDTH-1:0]   reg_data_out,
    output  [UDP_REG_SRC_WIDTH-1:0]      reg_src_out,

    // misc
    input                               reset,
    input                               clk
);

//----- Signals-----
/*REGISTER INTERFACE*/
```

```

//----- Signals-----
/*REGISTER INTERFACE*/
// software registers
wire [31:0] instr_addr;
wire [31:0] instr_data;
wire [31:0] data_addr;
wire [31:0] data_dataLow;
wire [31:0] data_dataHigh;
wire [31:0] datapath_cmd;
wire [31:0] mode;

// hardware registers
reg [31:0] data_low0;
reg [31:0] data_high0;

//Top-level Instruction memory signals
wire out_wr0;
wire in_rdy0;

// Defining instruction counters for four different threads, as we have a multithreading system.
reg [6:0] instructionCounter0; // Counter for thread 0, ranges from 0 to 127 (7 bits wide).
reg [7:0] instructionCounter1; // Counter for thread 1, ranges from 128 to 255 (8 bits wide).
reg [8:0] instructionCounter2; // Counter for thread 2, ranges from 256 to 383 (9 bits wide).
reg [8:0] instructionCounter3; // Counter for thread 3, ranges from 384 to 511 (9 bits wide).

// The actual program counter used in our datapath, reflecting the current instruction's address.
wire [8:0] PC;
wire [8:0] instrAddrIn; // Input to the instruction memory, indicating which instruction to fetch.

// Implementing a round-robin scheduling mechanism for the four threads.
reg [1:0] thread_num0; // 2-bit counter to track the current thread being executed (thread0, thread1, thread2, thread3).

// Signals related to instruction memory and instruction fetch stage.
wire [31:0] instrDataIn; // Data to be written into the instruction memory.
wire instrWeaIn; // Write enable signal for the instruction memory.
wire PCsel; // Selector for choosing the next program counter value.
wire [8:0] jump_address; // Address to jump to when executing a jump instruction.
wire [31:0] instrDataOut; // Data read from the instruction memory.

// Top level data memory signals for thread 0.
wire [31:0] dataDataOut_hi0; // Higher 32 bits of the data read from memory.
wire [31:0] dataDataOut_lo0; // Lower 32 bits of the data read from memory.

// Control signals for the processor's operation and inter-thread communication.
wire processor_control0; // Control signal specific for the processor's operation for thread 0.

// Determine the current program counter based on the active thread number.
assign PC = thread_num0[1] ? (thread_num0[0] ? instructionCounter3 : instructionCounter2)
: (thread_num0[0] ? {1'b0, instructionCounter1}
: {2'b0, instructionCounter0});

// Assign instruction address input based on mode selection.
assign instrAddrIn = mode[2] ? PC : (mode[1] ? 9'bX : instr_addr[8:0]);
// Assign instruction data input based on mode selection.
assign instrDataIn = mode[2] ? 32'bX : (mode[1] ? 32'bX : ( mode[0] ? instr_data : 32'bX ));
// Determine the write enable signal for instruction memory based on mode selection.
assign instrWeaIn = mode[2] ? 1'b0 : (mode[1] ? 1'b0 : ( mode[0] ? 1'b1 : 1'b0));

//----- Modules-----

bus_arbiter input_bus_arbiter (
    //EDIT 4/12 TO CHANGE REQUEST SIGNALS
    .req0(in_rdy0),
    .clk(clk),
    .rst(reset),
    .enable(),
    .gnt0(gnt0),
);

ids dp0 (
    .clk (clk),
    .reset (reset),
    .thread_num (thread_num0),
    .instrMem_addr (instrAddrIn),
    .instrMem_data (instrDataIn),
    .instrMem_wea (instrWeaIn),
    .dMem_addrTopLevel (data_addr[7:0]),
    .dMem_dataTopLevel ({data_dataHigh, data_dataLow}),
    .modeTopLevel (mode[2:0]),
    .input_grant(gnt0),
    .output_grant(gnt0_output),
    .in_data (in_data),
    .in_ctrl (in_ctrl),
    .in_wr (in_wr),
    .in_rdy (in_rdy0),
    .out_data (out_data0),
    .out_ctrl (out_ctrl0),
    .out_wr (out_wr0),
    .out_rdy (out_rdy),
    .thread_can_inc (thread_can_inc0), // used for incrementing thread_num
    .processor_control_out(processor_control0),
);

```

```

        .out_rdy (out_rdy),
        .thread_can_inc (thread_can_inc0), // used for incrementing thread_num
        .processor_control_out (processor_control0),
        .output_req (output_req0),
        .PCsel (PCsel),
        .jump_address (jump_address),
        .dMem_outHiTopLevel (dataDataOut_hi0),
        .dMem_outLoTopLevel (dataDataOut_lo0),
        .iMem_outTopLevel (instrDataOut)
    );

    bus_arbiter output_bus_arbiter (
        .req0 (output_req0),
        .clk (clk),
        .rst (reset),
        .gnt0 (gnt0_output),
    );

    assign out_wr = out_wr0;
    assign out_data = out_data0;
    assign out_ctrl = out_ctrl0;
    assign in_rdy = in_rdy0;

    generic_regs
    #(
        .UDP_REG_SRC_WIDTH (UDP_REG_SRC_WIDTH),
        .TAG ('BLOCK_ADDR), // Tag -- eg. MODULE_TAG
        .REG_ADDR_WIDTH ('REG_ADDR_WIDTH), // Width of block addresses -- eg. MODULE_REG_ADDR_WIDTH
        .NUM_COUNTERS (0), // Number of counters
        .NUM_SOFTWARE_REGS (7), // Number of sw regs
        .NUM_HARDWARE_REGS (4) // Number of hw regs
    ) module_regs (
        .reg_req_in (reg_req_in),
        .reg_ack_in (reg_ack_in),
        .reg_rd_wr_L_in (reg_rd_wr_L_in),
        .reg_addr_in (reg_addr_in),
        .reg_data_in (reg_data_in),
        .reg_src_in (reg_src_in),

        .reg_req_out (reg_req_out),
        .reg_ack_out (reg_ack_out),
        .reg_rd_wr_L_out (reg_rd_wr_L_out),
        .reg_addr_out (reg_addr_out),
        .reg_data_out (reg_data_out),
        .reg_src_out (reg_src_out),

        .reg_src_out (reg_src_out),

        // --- counters interface
        .counter_updates (),
        .counter_decrement(),

        // --- SW regs interface
        .software_regs ({mode}),

        // --- HW regs interface
        .hardware_regs ({data_low0, data_high0}),

        .clk (clk),
        .reset (reset)
    );

    //----- Logic-----

    always @(posedge clk) begin
        if(reset) begin
            // Set initial values for instruction counters for each thread.
            instructionCounter0 <= 7'b0; // Set for thread 0.
            instructionCounter1 <= 8'h80; // Set for thread 1, starts at 128.
            instructionCounter2 <= 9'h100; // Set for thread 2, starts at 256.
            instructionCounter3 <= 9'h180; // Set for thread 3, starts at 384.
            thread_num0 <= 2'b0; // Reset the thread counter to 0.
            data_low0 <= 32'b0; // Reset the lower data register.
            data_high0 <= 32'b0; // Reset the higher data register.
        end
        else begin
            // If datapath command indicates, reset data registers to zero.
            if (datapath_cmd[0]) begin
                data_low0 <= 32'b0;
                data_high0 <= 32'b0;
            end
            else begin
                // Data read mode: fetch data from memory and reset instruction counters and thread number.
                if(!mode[2]) && (mode[1]) && (!mode[0]) begin
                    data_low0 <= dataDataOut_lo0; // Load the lower part of data from memory.
                    data_high0 <= dataDataOut_hi0; // Load the higher part of data from memory.
                    // Reset instruction counters and thread number as it's a new cycle.
                    instructionCounter0 <= 7'b0;
                    instructionCounter1 <= 8'h80;
                    instructionCounter2 <= 9'h100;
                    instructionCounter3 <= 9'h180;
                    thread_num0 <= 2'b0;
                end
            end
        end
    end

```



```

// Instruction read mode: reset data registers, instruction counters, and thread number.
else if ((!mode[2]) && (!mode[1]) && (!mode[0])) begin
    data_high0 <= instrDataOut; // Store instruction data.
    data_low0 <= 32'b0; // Clear the lower data register.
    // Reset instruction counters and thread number to start values.
    instructionCounter0 <= 7'b0;
    instructionCounter1 <= 8'h80;
    instructionCounter2 <= 9'h100;
    instructionCounter3 <= 9'h180;
    thread_num0 <= 2'b0;
end
// Execution mode: check processor control and update counters accordingly.
else if (mode[2]) begin
    if (!processor_control0) begin // Processor is idle, reset counters and thread number.
        thread_num0 <= 2'b0;
        instructionCounter0 <= 7'b0;
        instructionCounter1 <= 8'h80;
        instructionCounter2 <= 9'h100;
        instructionCounter3 <= 9'h180;
    end
    else if (PCsel) begin // If PC selection is enabled, jump to specified address.
        // Update the instruction counter for the current thread based on jump_address.
        case(thread_num0)
            2'b11: instructionCounter3 <= jump_address; // Thread 3
            2'b10: instructionCounter2 <= jump_address; // Thread 2
            2'b01: instructionCounter1 <= jump_address[7:0]; // Thread 1
            default: instructionCounter0 <= jump_address[6:0]; // Thread 0
        endcase
    end
    else begin
        // Round-robin scheduling: increment the thread counter if allowed.
        if (thread_can_inc0) begin
            thread_num0 <= thread_num0 + 1'b1;
        end
        // Increment instruction counters based on the current thread and check for overflow.
        case(thread_num0)
            2'b11: // Thread 3
                instructionCounter3 <= (instructionCounter3 == 9'h1ff) ? 9'h180 : instructionCounter3 + 1'b1;
            2'b10: // Thread 2
                instructionCounter2 <= (instructionCounter2 == 9'h17f) ? 9'h100 : instructionCounter2 + 1'b1;
            2'b01: // Thread 1
                instructionCounter1 <= (instructionCounter1 == 8'hff) ? 8'h80 : instructionCounter1 + 1'b1;
            default: // Thread 0
                instructionCounter0 <= (instructionCounter0 == 7'h7f) ? 7'b0 : instructionCounter0 + 1'b1;
        endcase
    end
end
end
end
end // else: !if(reset)
end // always @(posedge clk)

endmodule

```

Bus_arbiter:

```
`timescale 1ns/1ps

// This is a two input round robin bus arbiter (with enable signal)
module bus_arbiter (
    input req0,
    input req1,
    input clk,
    input rst,
    // input enable,
    output gnt0,
    output gnt1
);
parameter GRANT0 = 1'b0;
parameter GRANT1 = 1'b1;

// The state determines which 'req' is granted.
// State '0' grants req0 and state '1' grants req1.
reg gnt_state;
reg next_state;

//assign gnt0 = req0 && !gnt_state && enable;
//assign gnt1 = req1 && gnt_state && enable;
assign gnt0 = req0 && !gnt_state;
assign gnt1 = req1 && gnt_state;

// The next state logic implements round robin fairness
// for two inputs. When both reqs are asserted, we grant req0 first;
```

```
always @(*) begin
    case (gnt_state)
        GRANT0: begin
            if (req0) begin
                next_state = 0;
            end else if (req1) begin
                next_state = 1;
            end else begin
                next_state = 0;
            end
        end
        GRANT1: begin
            if (req1) begin
                next_state = 1;
            end else begin
                next_state = 0;
            end
        end
    endcase
end // always @(*)
```

```
// State registers with asynchronous reset
always @(posedge clk, posedge rst) begin
    if (rst) begin
        gnt_state <= 0;|
```

Thread0:

```
#include <stdint.h>
#include <stdio.h>
#include <unistd.h>

uint32_t stateVariable = 0;
uint32_t readPointer = 0;
uint32_t currentPacketComplete = 0;
uint32_t ParentThread = 0;

extern uint32_t Thread1Complete;
extern uint32_t Thread2Complete;
extern uint32_t Thread3Complete;

void Thread0(void);

void Thread0(void) {

    const uint32_t desiredState = 3;
    const uint32_t udpProtocol = 17;

    while (1) {
        if (stateVariable == desiredState) {
            if ((readPointer & 0xFF) == udpProtocol) {
                ParentThread = 0;

                while (Thread1Complete == 0 || Thread2Complete == 0 || Thread3Complete == 0) {
                    sleep(1);
                }

                printf("Thread 0: All child Threads completed their tasks.\n");
                currentPacketComplete = 1;

                Thread1Complete = Thread2Complete = Thread3Complete = 0;
                ParentThread = 0;
            }
        }
        sleep(1);
    }
}

int main(void) {

    Thread0();

    return 0;
}
```

Thread1:

```
#include <stdint.h>
#include <stdio.h>

extern volatile uint32_t ParentThread;
extern uint32_t readPointer;
uint32_t Thread1Complete = 0;

void Thread1(void);
void make_IP_checksum_zero(void);

void Thread1(void) {
    const uint32_t signalToStart = 0;

    while(1) {
        if (ParentThread == signalToStart) {
            make_IP_checksum_zero();
            Thread1Complete = 1;
            break;
        }
    }
}

void make_IP_checksum_zero(void) {
    printf("Thread 1: Making IP checksum zero.\n");
    printf("IP checksum has been set to zero.\n");
}

int main(void) {
    ParentThread = 0;
    readPointer = 512;
    ParentThread = 0;
    Thread1();
    return 0;
}
```


Thread2:

```
#include <stdio.h>
#include <stdint.h>

volatile uint32_t ParentThread = 0;
volatile uint32_t completionStatusThread2 = 0;
uint32_t packetLocation = 512;
uint32_t PacketStatusLocation = 1024;

void Thread2(void);

void Thread2(void) {

    const uint32_t parentSignal = 0;
    uint64_t packetData;

    while(1) {
        if (ParentThread == parentSignal) {

            packetData = *((uint64_t*)packetLocation);

            packetData &= ~(uint64_t)0;

            *((uint64_t*)packetLocation) = packetData;

            completionStatusThread2 = 2;

            ParentThread = 0;

        }
    }
}

int main() {

    ParentThread = 0;

    Thread2();

    return 0;
}
```

Thread3:

```
#include <stdint.h>
#include <stdio.h>

extern volatile uint32_t ParentThread;
uint32_t Thread3Complete = 0;
uint32_t currentPacketComplete = 0;

void Thread3(void);
void manipulate_payload(void);

void Thread3(void) {
    const uint32_t signalToStart = 0;

    while(1) {
        if (ParentThread == signalToStart) {
            manipulate_payload();

            ParentThread = 0;
            Thread3Complete = 1;
            break;
        }
    }
}

void manipulate_payload(void) {
    printf("Thread 3: Manipulating payload data.\n");

    uint32_t* payloadDataLocation = (uint32_t*)512;
    uint32_t encodedValue;

    encodedValue = 4;

    *payloadDataLocation = encodedValue;

    printf("Payload manipulation completed.\n");

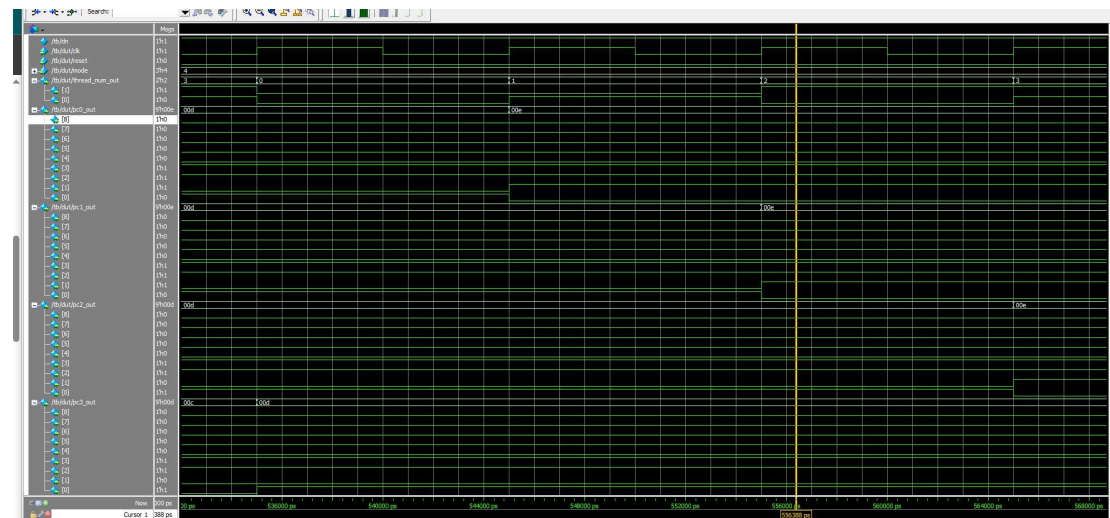
    currentPacketComplete = 1;
}

int main(void) {
    ParentThread = 0;

    Thread3();

    return 0;
}
```

Multiple thread's waveform.



```
thr=1 pc0= 10 pc1= 9 pc2= 9 pc3= 9 w0=0 w1=1 w2=0 w3=0 done=1
thr=1 pc0= 10 pc1= 9 pc2= 9 pc3= 9 w0=0 w1=1 w2=0 w3=0 done=1
thr=2 pc0= 10 pc1= 10 pc2= 9 pc3= 9 w0=0 w1=0 w2=1 w3=0 done=1
thr=2 pc0= 10 pc1= 10 pc2= 9 pc3= 9 w0=0 w1=0 w2=1 w3=0 done=1
thr=3 pc0= 10 pc1= 10 pc2= 10 pc3= 9 w0=0 w1=0 w2=0 w3=1 done=1
thr=3 pc0= 10 pc1= 10 pc2= 10 pc3= 9 w0=0 w1=0 w2=0 w3=1 done=1
thr=0 pc0= 10 pc1= 10 pc2= 10 pc3= 10 w0=1 w1=0 w2=0 w3=0 done=1
thr=0 pc0= 10 pc1= 10 pc2= 10 pc3= 10 w0=1 w1=0 w2=0 w3=0 done=1
thr=1 pc0= 11 pc1= 10 pc2= 10 pc3= 10 w0=0 w1=1 w2=0 w3=0 done=1
thr=1 pc0= 11 pc1= 10 pc2= 10 pc3= 10 w0=0 w1=1 w2=0 w3=0 done=1
thr=2 pc0= 11 pc1= 11 pc2= 10 pc3= 10 w0=0 w1=0 w2=1 w3=0 done=1
thr=2 pc0= 11 pc1= 11 pc2= 10 pc3= 10 w0=0 w1=0 w2=1 w3=0 done=1
thr=3 pc0= 11 pc1= 11 pc2= 11 pc3= 10 w0=0 w1=0 w2=0 w3=1 done=1
thr=3 pc0= 11 pc1= 11 pc2= 11 pc3= 10 w0=0 w1=0 w2=0 w3=1 done=1
thr=0 pc0= 11 pc1= 11 pc2= 11 pc3= 11 w0=1 w1=0 w2=0 w3=0 done=1
thr=0 pc0= 11 pc1= 11 pc2= 11 pc3= 11 w0=1 w1=0 w2=0 w3=0 done=1
```

This waveform confirms that the multi-threaded context switching is working correctly. The **thread_num_out** signal cycles through 0, 1, 2, and 3, showing the processor is switching among the four threads. Each **pc_out** increments only when its corresponding thread is active, and remains stable otherwise. Finally, **done** asserts once a certain count is reached, demonstrating that each thread runs independently and the context switching behaves as intended.

NetFpga:

```
Found net device: nf2c0
nf_download /home/netfpga/bitfiles/lab9.bit
[team-4:fpga ~] regread 0x2000300 0x2000304 0x2000308 0x200030C
Bit file built from: nf2_top_par.ncd;HW_TIMEOUT=FALSE
Part: 2vp50ff1152
Date: 2025/ 3/15
Time: 21:40:19
Error Registers: 0
Good, after resetting programming interface the FIFO is empty
Download completed - 2377668 bytes. (expected 2377668).
DONE went high - chip has been successfully programmed.
CPCI Information
-----
Version: 4 (rev 1)

Device (Virtex) Information
-----
Project directory: lab9
Project name: multi-thread
Project description: Integration of Hardware for Multiple Threads

Device ID: 102
Version: 0.1.0
Built against CPCI version: 4 (rev 1)

Virtex design compiled against active CPCI version

Virtex design compiled against active CPCI version
[team-4:fpga ~] rkd&
[1] 8849
[team-4:fpga ~] regread 0x2000300 0x2000304 0x2000308 0x200030C
Found net device: nf2c0
Reg 0x02000300 (33555200): 0x00000000 (0)
Reg 0x02000304 (33555204): 0x00000000 (0)
Reg 0x02000308 (33555208): 0x00000000 (0)
Reg 0x0200030c (33555212): 0xdeadbeef (3735928559)

-----
Client connecting to 10.0.17.3, UDP port 4800
Sending 1470 byte datagrams
UDP buffer size: 2.00 MByte (default)
-----
[ 3] local 10.0.17.3 port 4800 connected with 10.0.16.3 port 4399
-----
Server listening on UDP port 4800
Receiving 1470 byte datagrams
UDP buffer size: 2.00 MByte (default)
-----
[ 4] local 10.0.16.3 port 4399 connected with 10.0.17.3 port 4800

Receiving 1470 byte datagrams
UDP buffer size: 2.00 MByte (default)
-----
[ 4] local 10.0.16.3 port 4399 connected with 10.0.17.3 port 4800

[ ID] Interval      Transfer      Bandwidth      Jitter  Lost/Total Datagrams
[ 3] 0.0- 0.0 sec  1.44 KBytes   978 Kbits/sec   0.000 ms    0/    1 (0%)
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.0- 0.0 sec  1.44 KBytes   916 Kbits/sec
[ 4] Sent 1 datagrams
[ 4] Server Report:
[ 4] 0.0- 0.0 sec  1.44 KBytes   978 Kbits/sec   0.000 ms    0/    1 (0%)
[team-4:fpga ~] regread 0x2000300 0x2000304 0x2000308 0x200030C
Found net device: nf2c0
Reg 0x02000300 (33555200): 0x00000000 (0)
Reg 0x02000304 (33555204): 0x00000004 (4)
Reg 0x02000308 (33555208): 0x00000000 (0)
Reg 0x0200030c (33555212): 0xdeadbeef (3735928559)
```